Code Layout, Readability and Reusability

Wandr AI – AI Travel Itinerary Planner

1. Backend Code Layout

Module	Location	Responsibility	LOC (approx)
config.py	app/core/	pydantic-settings; all env vars; computed properties	80
security.py	app/core/	JWT decode via JWKS or HS256; get_current_user dependency	120
main.py	app/	FastAPI app init; CORSMiddleware with allow_origin_regex	100
trips.py	app/api/routes/	All /api/trips/* route handlers	204
health.py	app/api/routes/	GET /api/health – configuration status	22
ai_service.py	app/services/	Groq prompt building, API call, JSON parsing, retry logic	371
itinerary_service.py	app/services/	Supabase CRUD for trips	~300
budget_service.py	app/services/	Cost aggregation and budget breakdown	~150
trip.py	app/schemas/	All Pydantic models (request, response, enums)	~250

2. Frontend Code Layout

Module / File	Location	Responsibility
main.tsx	src/	React root; BrowserRouter; all routes defined
AuthContext.tsx	src/contexts/	Supabase session state; AuthProvider; useAuth hook
api.ts	src/lib/	ApiClient class; all fetch calls with Bearer JWT

supabase.ts	src/lib/	Supabase JS client initialisation (anon key)
LandingPage	src/pages/	Marketing page; value proposition
CreateTripPage	src/pages/	Trip creation form (React Hook Form + Zod)
TripDetailPage	src/pages/	Itinerary display; day cards; Budget tab
TripsListPage	src/pages/	Paginated trips list; search
BudgetPanel	src/components/trip/	Recharts budget visualisation

3. Readability Standards Applied

• All Python modules have Google-style docstrings on public functions.

• FastAPI routes use typed Pydantic models for request and response — no untyped dicts in API contracts.

• TypeScript strict mode enabled; no implicit 'any'.

• Environment variable names are documented in .env.example and README.

• Constants (e.g., max_tokens=8000, temperature=0.7) are explicit in ai_service.py — not magic numbers.

4. Reusability Patterns

• **ApiClient class (api.ts):** A single class handles all backend calls, attaches JWT, and standardises error handling — frontend pages import and call it; they don't write fetch() directly.

• **get_current_user dependency (security.py):** FastAPI dependency injection; any route that needs auth simply adds current_user: dict = Depends(get_current_user).

• **_call_groq() with @retry decorator:** Groq API call logic is isolated and reused by both generate_itinerary() and regenerate_day().

• **_parse_day() helper:** Parses a raw day dict into an ItineraryDay Pydantic model — shared by full generation and single-day regeneration.